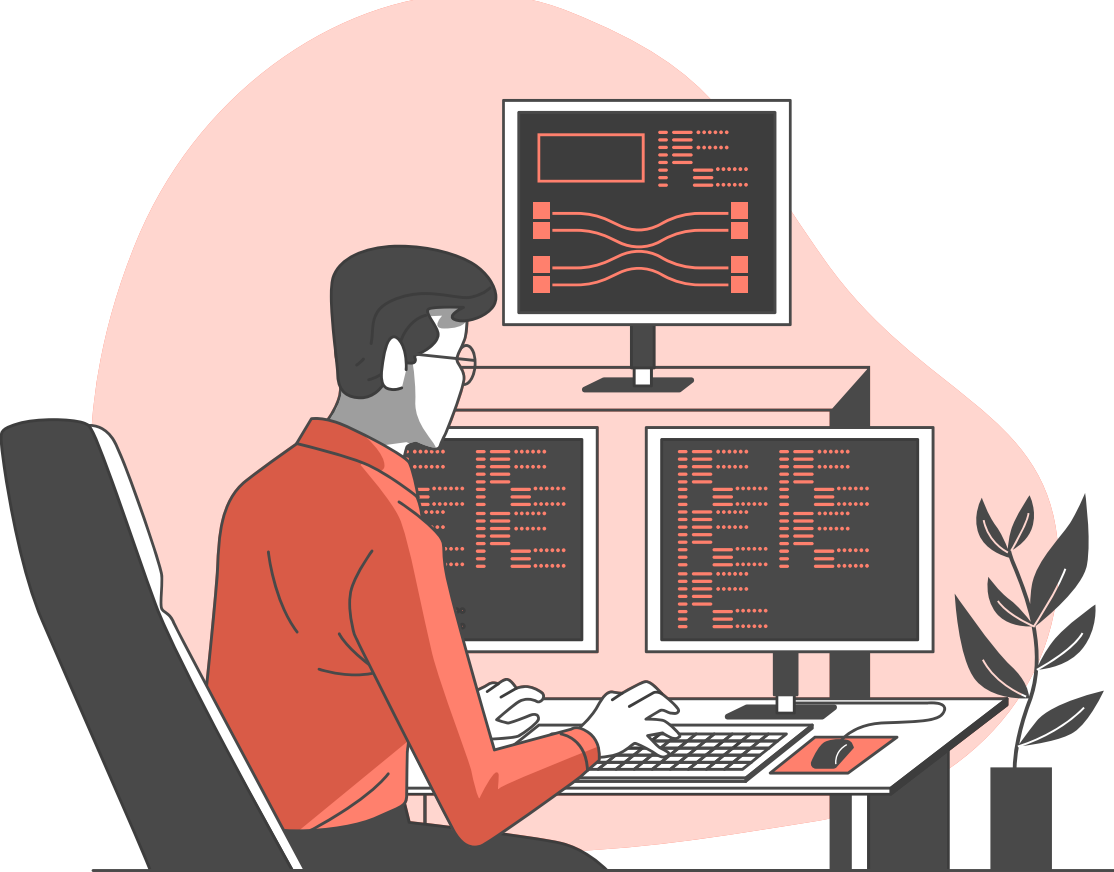




Pre-Transformer Models → Transformer

Gizem Gezici (RTDA)

Date: 12.05.2026



Funded by
the European Union
NextGenerationEU



Ministero
dell'Università
e della Ricerca



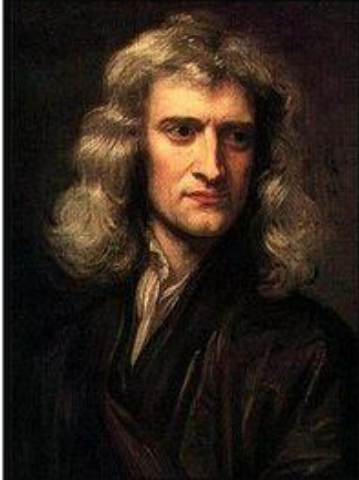
Italiani domani
PIANO NAZIONALE
DI RIPRESA E RESILIENZA



Future
Artificial
Intelligence
Research



LLMs were not built **overnight**



If I have seen further than others, it is by standing
upon the shoulders of giants.

(Isaac Newton)

izquotes.com

Image Source: <https://shorturl.at/Xtfwa>

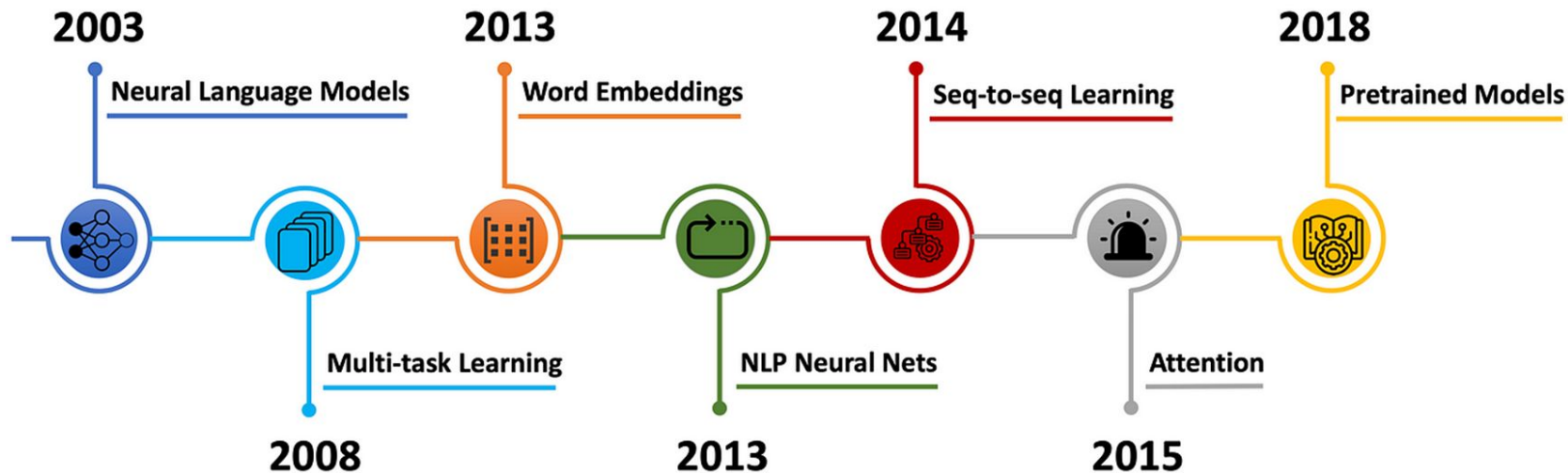
Ref: <https://digestize.medium.com/a-brief-history-of-nlp-leading-to-llms-part-1-915ed263cbfa>

Outline

- **From Words to Meaning in Context**
- **Modeling Sequences (RNNs & Seq2Seq)**
- **Attention Mechanism**
- **Transformer Architecture**
- **From Transformers to Modern LLMs**
- **Summary and Next Steps**

NLP Evolution (in 2000s)

👉 from counting (Ngrams) → embeddings → sequences → attention → LLMs



Meaning from Context (Firth, 1957)



“You shall know a word
by the company it keeps!”

*J. R. Firth,
A synopsis of linguistic theory (1957)*



Key idea:

Words are defined by their **context**

Can we understand a word without knowing it?

What does **wampimuk** mean?

Meaning comes from **context**

What does wampimuk mean?

Marco saw a **hairy little** *wampimuk* **crouching behind a tree.**

N-gram Language Model

- Predict the next word based on previous words
- Uses local context (previous $n - 1$)

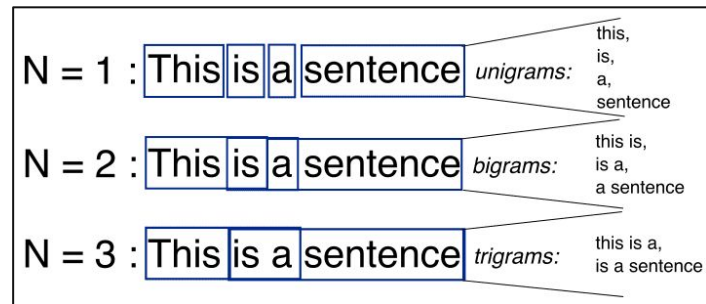
$$P(w_t | w_{t-1}, w_{t-2}, \dots, w_{t-n+1})$$

Probability of the next word given the previous $n - 1$ words

- Unigram: 1 word
- Bigram: 2 words
- Trigram: 3 words

 **Key idea:**

- Language modeled using **word sequences**



From Words to Meaning in Context

1. **N-grams** → Limited Context

- Predict next word using local window
- Only uses a short local context (last $n-1$ words)

👉 Problem:

- No long-range understanding
- Data sparsity

👉 Language depends on long-range context

Neural Language Model (Bengio, 2003)

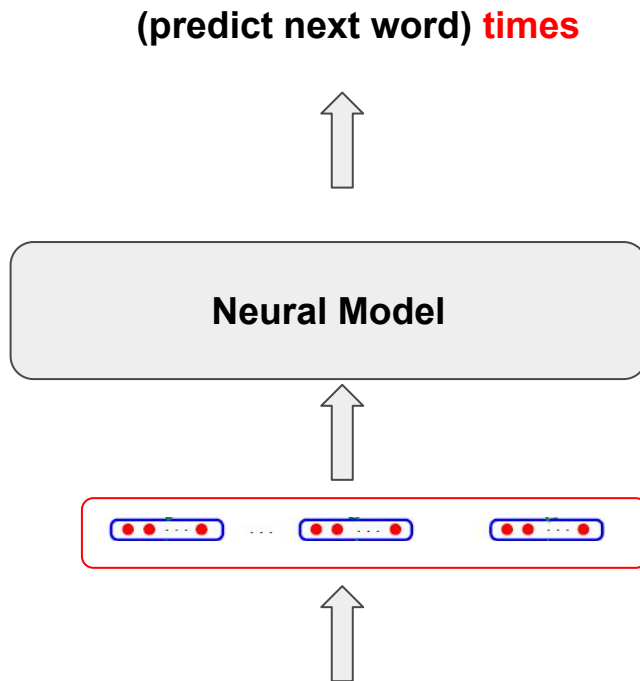
- **Fixed-context** Neural Language Model

👉 Same objective as N-gram:

$$P(w_t | w_{t-1}, w_{t-2}, \dots, w_{t-n+1})$$

- How does the model work?
 - **Input:** Vector representations of the previous $n - 1$ words
 - **Hidden:** Embeddings are fed into a neural network
 - **Output:** Softmax layer will predict the **next word** (probability distribution over the vocabulary)

👉 Key idea: **Learn word representations (embeddings) automatically**



It is a great movie, I have seen it several ____

Word2Vec (Mikolov et al., 2013)

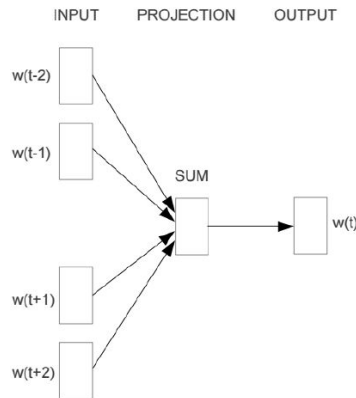
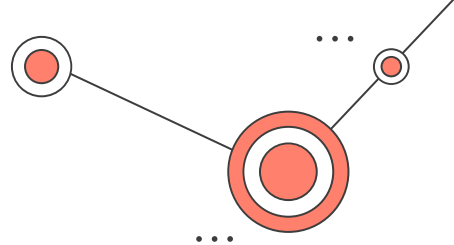
👉 Focus on learning embeddings efficiently
(not full language modeling)

- Learns word embeddings from context
- **CBOW**: predict target word from context
- **Skip-gram**: predict context words from target word
- Much faster and more scalable than neural language models

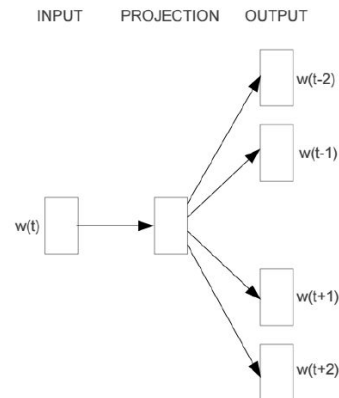
👉 Unlike Bengio:

- Not a full language model
- Focuses only on learning word embeddings

2013

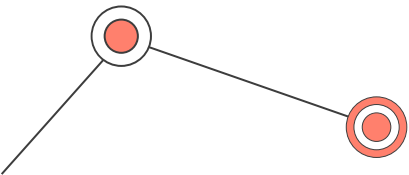


CBOW



Skip-gram

Mikolov et al., 2013



From Words to Meaning in Context

2. Neural Language Models → Word Embeddings

- Words are represented as continuous vectors
- Words → dense vectors (embeddings)
- Capture semantic similarity:
 - king $\approx$ queen
 - dog $\approx$ cat

From Words to Meaning in Context

2. Neural Language Models → Word Embeddings

- Words are represented as continuous vectors
- Words → dense vectors (embeddings)
- Capture semantic similarity:
 - king $\approx$ queen
 - dog $\approx$ cat

! BUT — Still a Problem

- Still **fixed window context**
- Still limited like N-grams (fixed context window)

From Words to Meaning in Context

2. Neural Language Models → Word Embeddings

- Words are represented as continuous vectors
- Words → dense vectors (embeddings)
- Capture semantic similarity:
 - king $\approx$ queen
 - dog $\approx$ cat

! BUT — Still a Problem

- Still **fixed window context**
- Still limited like N-grams (fixed context window)

👉 We improved word meaning, but not sequence understanding

From Words to Meaning in Context

3. Why Embeddings Are Not Enough

 Key limitations:

- No full sentence understanding
- No word order information (in methods like Word2Vec)
- No long dependencies

● **Example:**

- The cat chased the dog vs
- The dog chased the cat

Same words → completely different meaning

From Words to Meaning in Context

3. Why Embeddings Are Not Enough

 Key limitations:

- No full sentence understanding
- No word order information (in methods like Word2Vec)
- No long dependencies

● **Example:**

- The cat chased the dog vs
- The dog chased the cat

Same words → **completely different meaning**

 **Meaning depends on word order and context**

From Static to Contextual Embeddings

- Word2Vec:
 - One vector per word ✗
- ELMo (Embeddings from Language Models):
 - Context-dependent vectors ✓

Example:

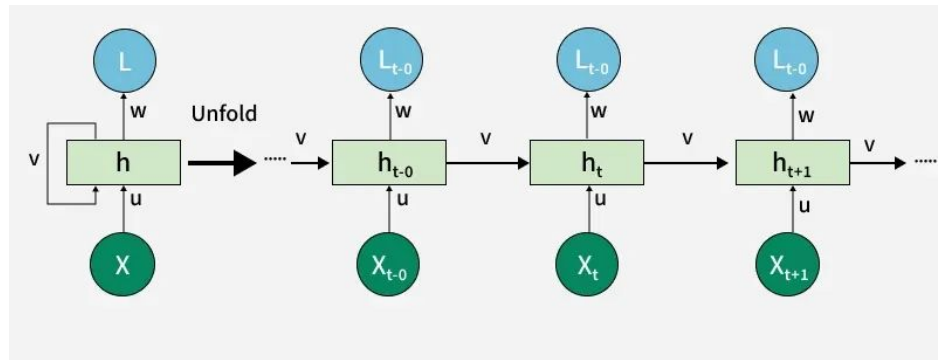
- “bank” (river vs finance)

What is missing?

We improved how we represent words,
but we still **do not understand sentences**.

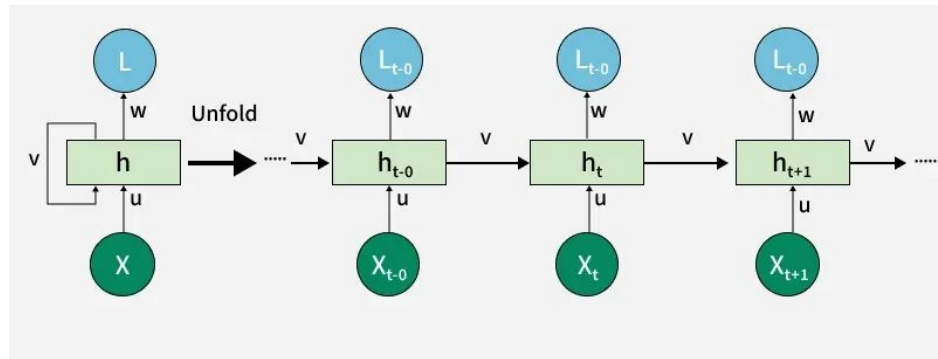
Recurrent Neural Networks (RNNs): Modeling Sequences

- Process text **one word at a time**
- Maintain a **hidden state (memory)**
- Capture **context** from previous words over time

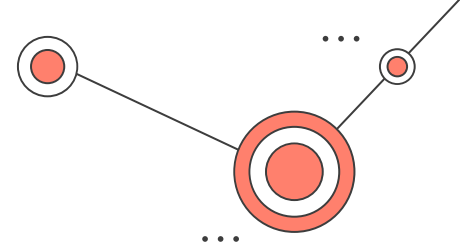


Recurrent Neural Networks (RNNs): Modeling Sequences

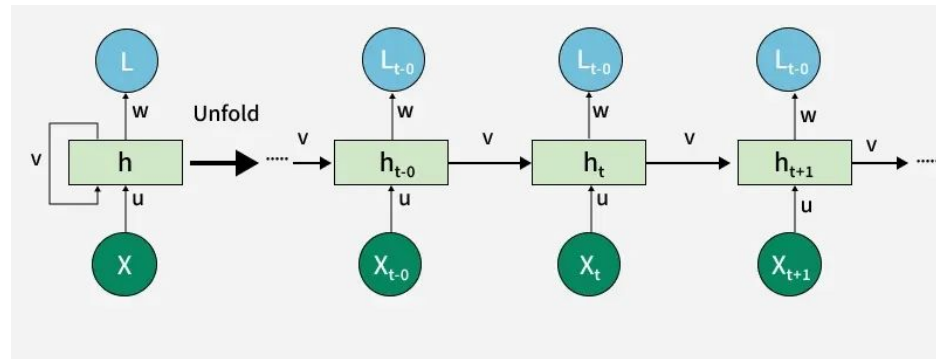
- Process text **one word at a time**
 - Maintain a **hidden state (memory)**
 - Capture **context** from previous words over time
- ❖ The entire past is compressed into a **fixed-size memory (hidden state)**



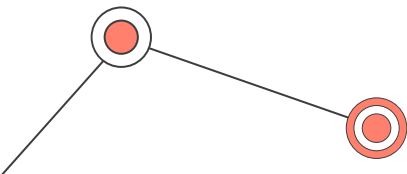
Recurrent Neural Networks (RNNs): Modeling Sequences



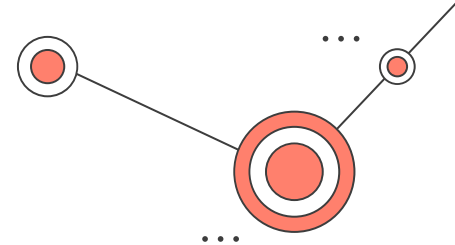
- Process text **one word at a time**
 - Maintain a **hidden state (memory)**
 - Capture **context** from previous words over time
- ❖ The entire past is compressed into a **fixed-size memory (hidden state)**



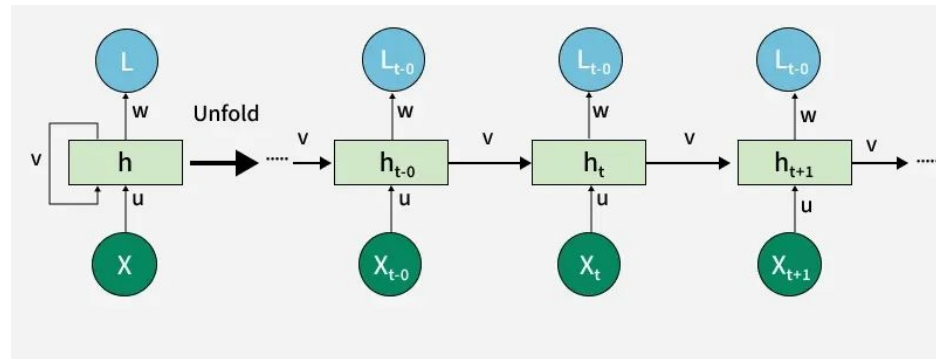
👉 This allows the model to capture order and context



Recurrent Neural Networks (RNNs): Modeling Sequences

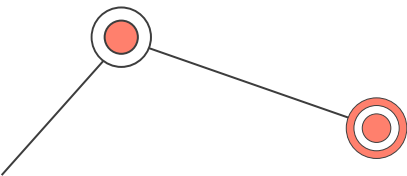


- Process text **one word at a time**
 - Maintain a **hidden state (memory)**
 - Capture **context from previous words** over time
- ❖ **The entire past is compressed into a fixed-size memory (hidden state)**



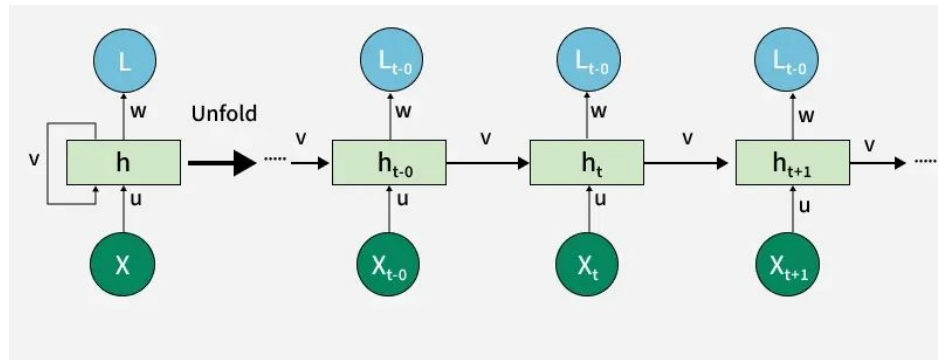
👉 **Limitation: Important information from earlier words may be forgotten**

- ... I grew up in France ... I speak fluent ____

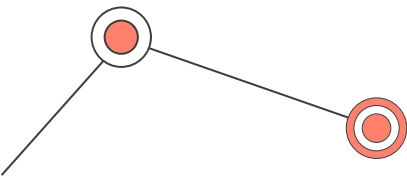


Recurrent Neural Networks (RNNs): Modeling Sequences

- Process text **one word at a time**
 - Maintain a **hidden state (memory)**
 - Capture **context** from previous words over time
- ❖ The entire past is compressed into a **fixed-size memory (hidden state)**



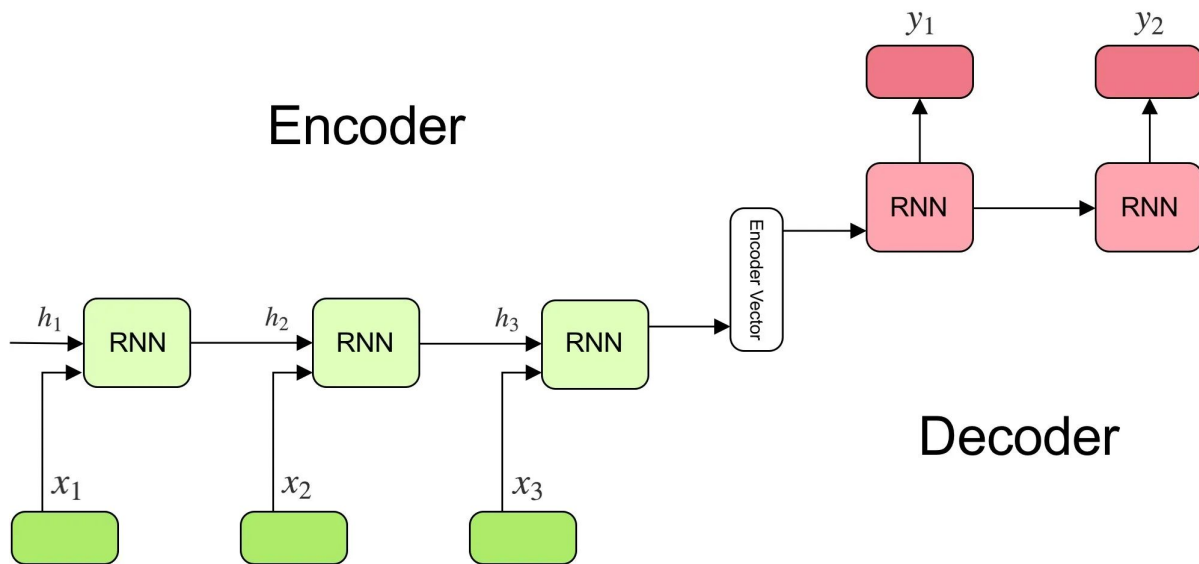
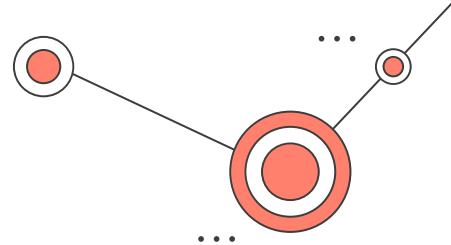
👉 Long-range dependencies are hard to capture



Encoder-Decoder

- Encoder reads the input $\rightarrow$ produces a **fixed-size** representation $\rightarrow$ decoder generates output

2014



<https://medium.com/analytics-vidhya/encoders-decoders-sequence-to-sequence-architecture-5644efbb3392>

Transformer Architecture

- Sequence-to-sequence translation
- Attention mechanism

Vaswani et al., Attention is all you need

Encoder

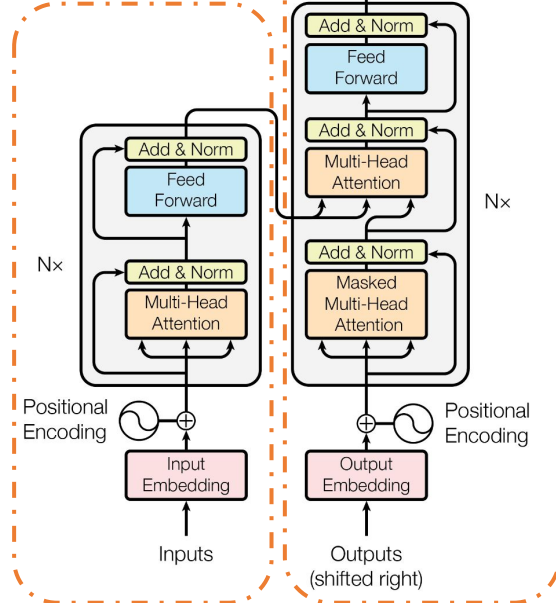


Figure 1: The Transformer - model architecture.

2017

Decoder

Two steps back : Sequence-to-Sequence Models

2014

- Sequence-to-sequence translation
- ~~Attention mechanism~~

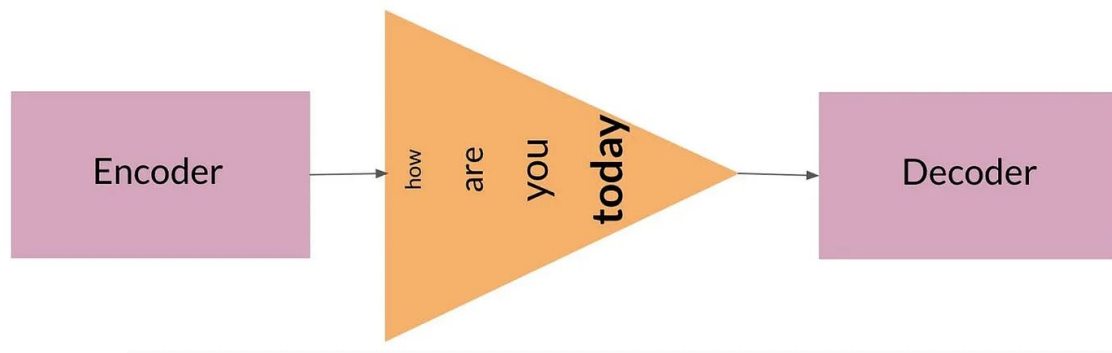


Image: <https://towardsdatascience.com/attention-and-its-different-forms-7fc3674d14dc>

Image: <https://jalamar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/>

Last hidden state = summary of the whole sentence (context vector)

! Problem: Single vector must represent the entire sentence → information bottleneck

[Sutskever et al., "Sequence to Sequence Learning with Neural Networks"](#)

[Cho, Kyunghyun, et al. "Learning phrase representations using RNN encoder-decoder for statistical machine translation."](#)

Two steps back : Sequence-to-Sequence Models

2014

- Sequence-to-sequence translation
- ~~Attention mechanism~~

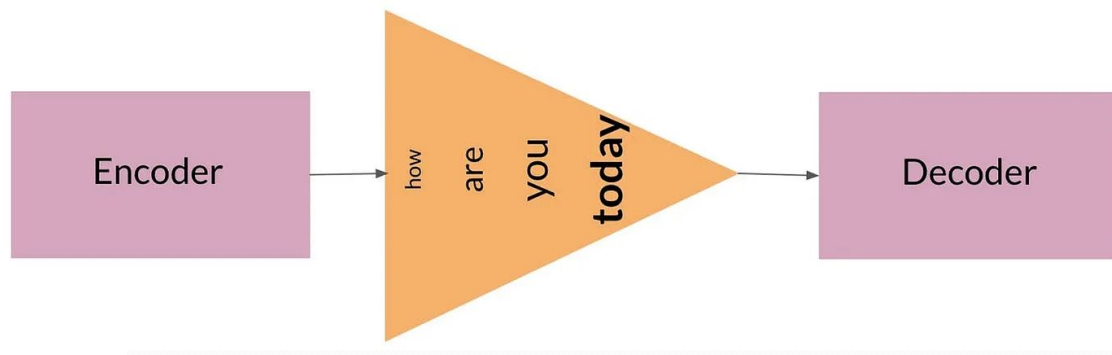


Image: <https://towardsdatascience.com/attention-and-its-different-forms-7fc3674d14dc>

Image: <https://jalamar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/>

Last hidden state = summary of the whole sentence (context vector)

! Problem: Single vector must represent the entire sentence → information bottleneck

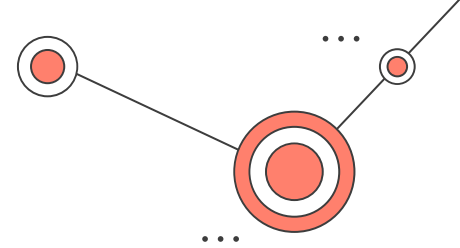
Better results: Researchers showed that *giving the source sentence* in a reverse order OR *giving it twice*

[Sutskever et al., "Sequence to Sequence Learning with Neural Networks"](#)

[Cho, Kyunghyun, et al. "Learning phrase representations using RNN encoder-decoder for statistical machine translation."](#)

One step back : Sequence-to-Sequence Models with Attention

2014



- Sequence-to-sequence translation
- Attention mechanism

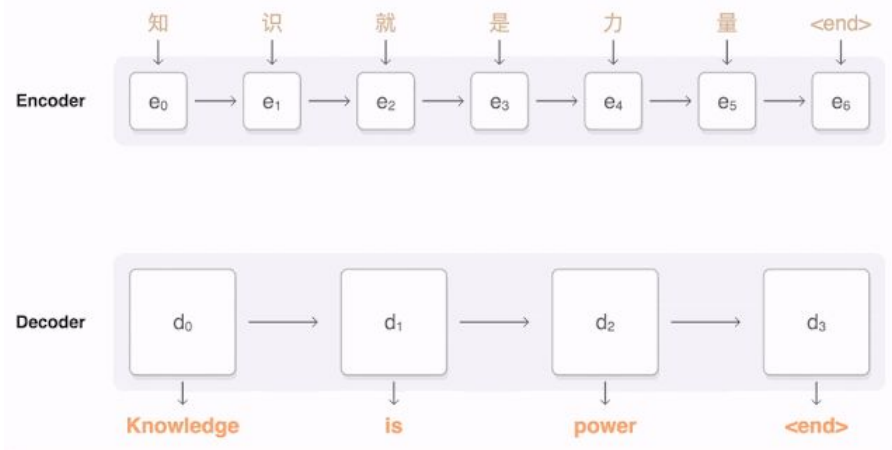
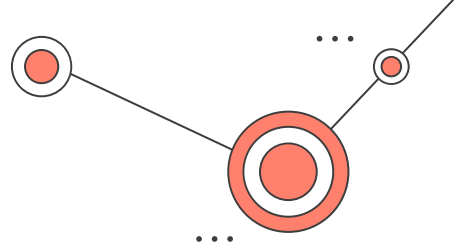


Image: <https://research.google/blog/a-neural-network-for-machine-translation-at-production-scale/>

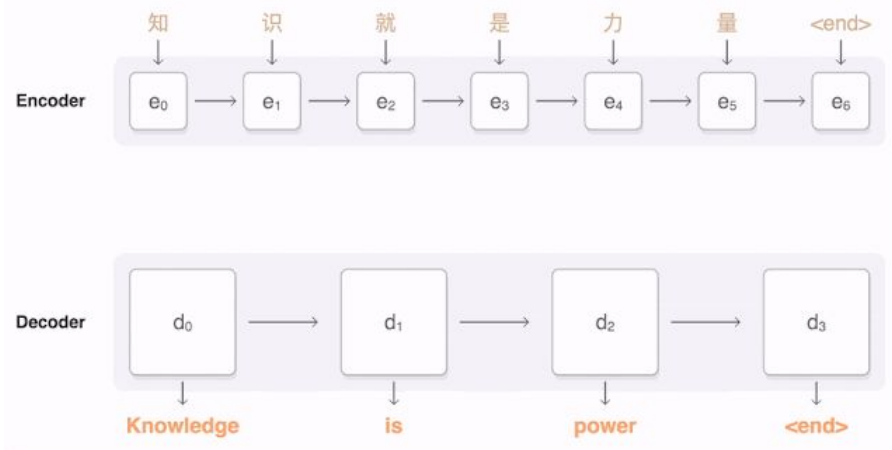
Bahdanau et al., 2014
Luong et al., 2015

One step back : Sequence-to-Sequence Models with Attention

2014



- Sequence-to-sequence translation
- Attention mechanism



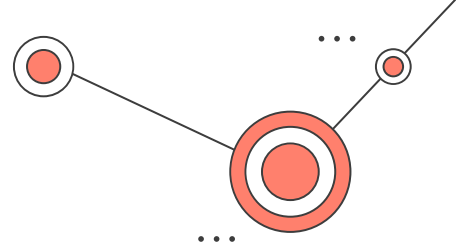
- Encoder produces representations for **all words**

Image: <https://research.google/blog/a-neural-network-for-machine-translation-at-production-scale/>

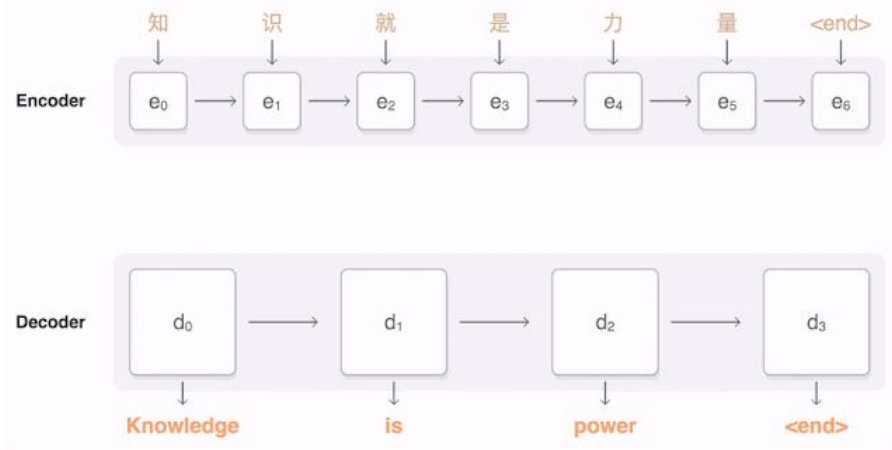
[Bahdanau et al., 2014](#)
[Luong et al., 2015](#)

One step back : Sequence-to-Sequence Models with Attention

2014



- Sequence-to-sequence translation
- Attention mechanism



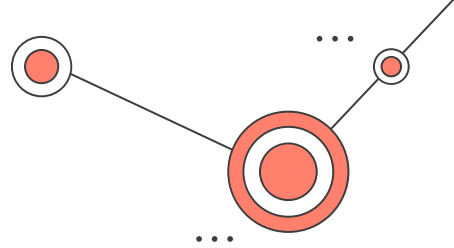
- Encoder produces representations for **all words**
- Decoder looks at **different parts of the input**

Image: <https://research.google/blog/a-neural-network-for-machine-translation-at-production-scale/>

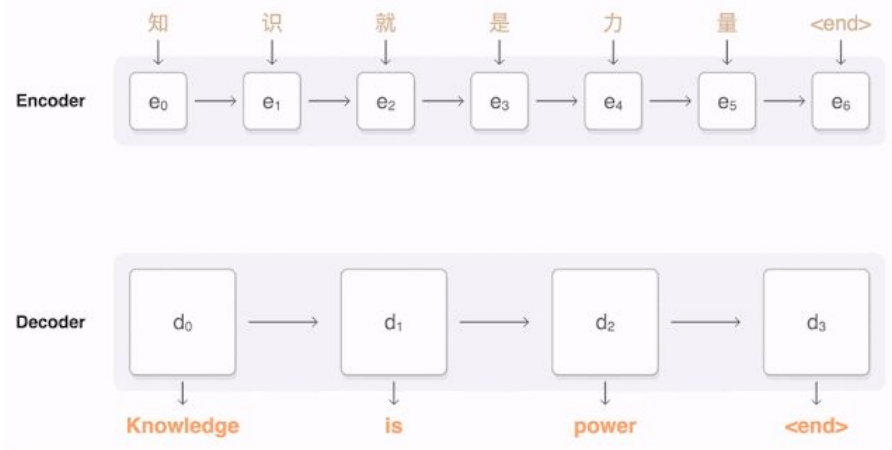
[Bahdanau et al., 2014](#)
[Luong et al., 2015](#)

One step back : Sequence-to-Sequence Models with Attention

2014



- Sequence-to-sequence translation
- Attention mechanism



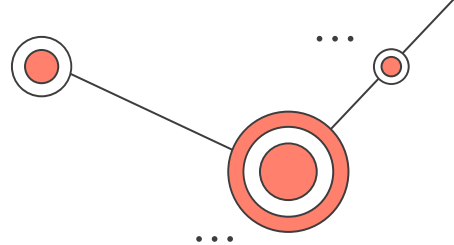
- Encoder produces representations for **all words**
- Decoder looks at **different parts of the input**
- Learns **what to focus on at each step**

Image: <https://research.google/blog/a-neural-network-for-machine-translation-at-production-scale/>

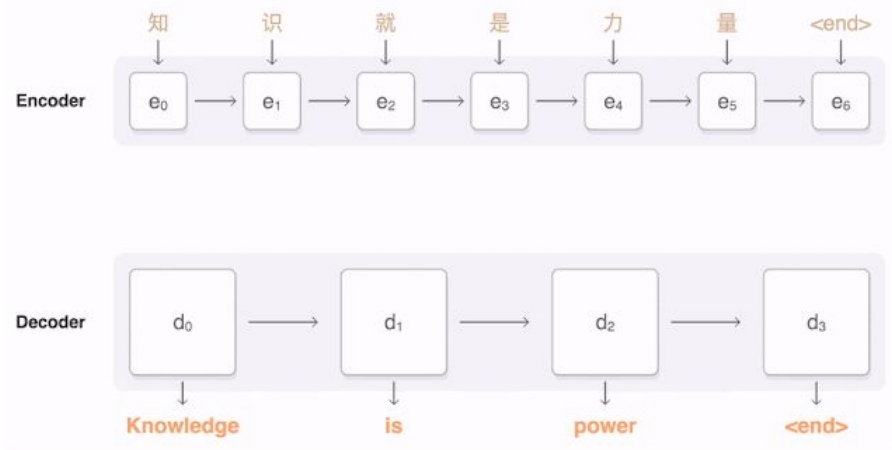
[Bahdanau et al., 2014](#)
[Luong et al., 2015](#)

One step back : Sequence-to-Sequence Models with Attention

2014



- Sequence-to-sequence translation
- Attention mechanism



- Encoder produces representations for **all words**
- Decoder looks at **different parts of the input**
- Learns **what to focus on at each step**

👉 Different words matter at different times

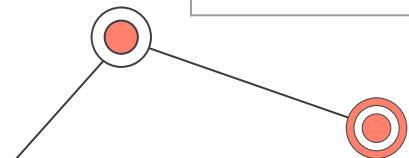
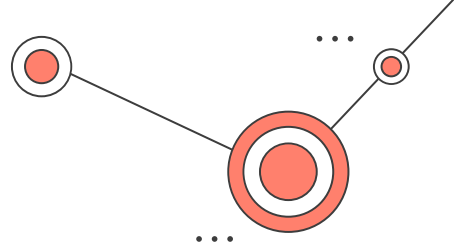
Image: <https://research.google/blog/a-neural-network-for-machine-translation-at-production-scale/>

[Bahdanau et al., 2014](#)
[Luong et al., 2015](#)

Example: How Attention Works

- **Translation (Italian → English)**
- **Input:** lo mangio una mela
- **Output** (generated step by step):
 - (The model dynamically shifts its attention)

Output word	Attention focus
I	lo
eat	mangio
an	una
apple	mela



[Bahdanau et al., 2014](#)
[Luong et al., 2015](#)

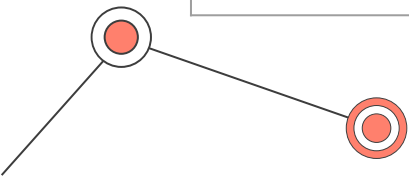
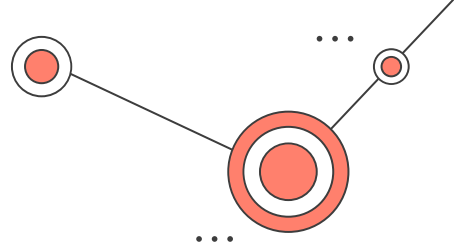
Example: How Attention Works

- **Translation (Italian → English)**
- **Input:** lo mangio una mela
- **Output** (generated step by step):
 - (The model dynamically shifts its attention)

Output word	Attention focus
I	lo
eat	mangio
an	una
apple	mela

👉 At each step, different words become important.

[Bahdanau et al., 2014](#)
[Luong et al., 2015](#)



Example: How Attention Works

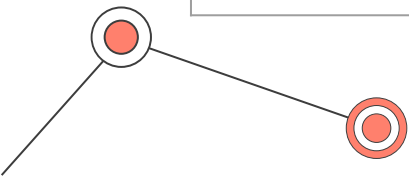
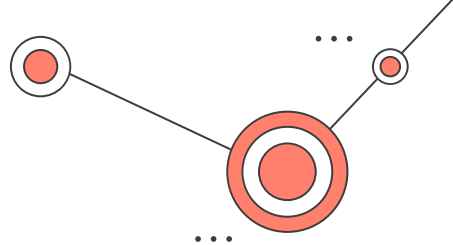
- **Translation (Italian → English)**
- **Input:** lo mangio una mela
- **Output** (generated step by step):
 - (The model dynamically shifts its attention)

Output word	Attention focus
I	lo
eat	mangio
an	una
apple	mela

👉 At each step, different words become important.

👉 The model doesn't treat all words equally—it focuses on what is relevant.

[Bahdanau et al., 2014](#)
[Luong et al., 2015](#)



One step back : Sequence-to-Sequence Models with Attention

- Sequence-to-sequence translation
- Attention mechanism

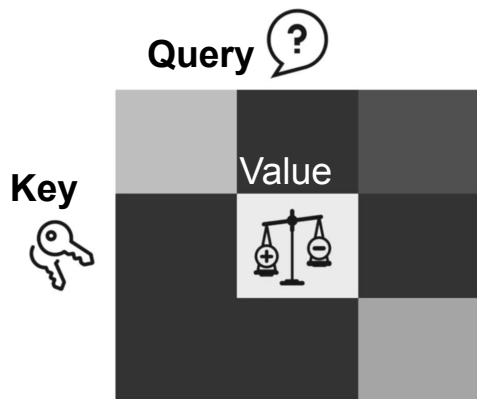
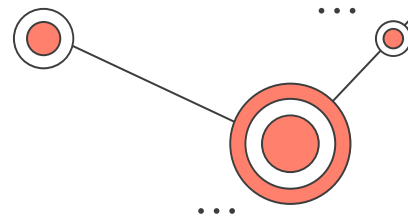


Image: <https://www.coursera.org/learn/attention-models-in-nlp/lecture/87aPC/attention>

[Bahdanau et al., 2014](#)
[Luong et al., 2015](#)

2014



Each vector receives three representations (“roles”)

$[W_Q] \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{blue} \\ \text{blue} \\ \text{blue} \end{bmatrix}$ **Query:** vector **from** which the attention is looking
“Hey there, do you have this information?”

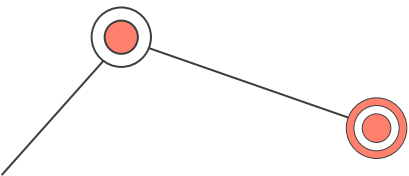
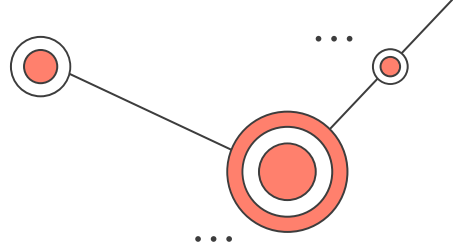
$[W_K] \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{yellow} \\ \text{yellow} \\ \text{yellow} \end{bmatrix}$ **Key:** vector **at** which the query looks to compute weights
“Hi, I have this information – give me a large weight!”

$[W_V] \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{red} \\ \text{red} \\ \text{red} \end{bmatrix}$ **Value:** their weighted sum is attention output
“Here’s the information I have!”

Image: https://lena-voita.github.io/nlp_course/seq2seq_and_attention.html

With Attention

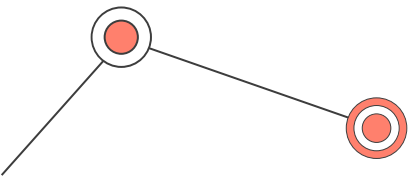
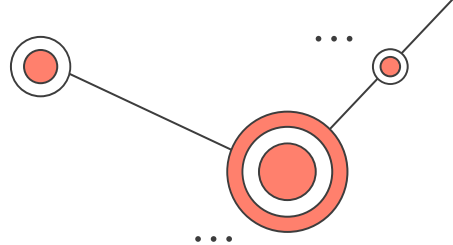
- We solved the bottleneck problem...
 - **but we still process words one by one (sequentially).**



With Attention

- We solved the bottleneck problem...
 - **but we still process words one by one (sequentially).**

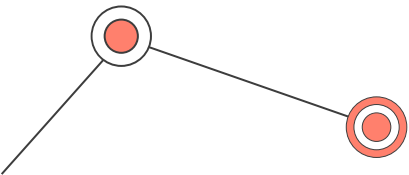
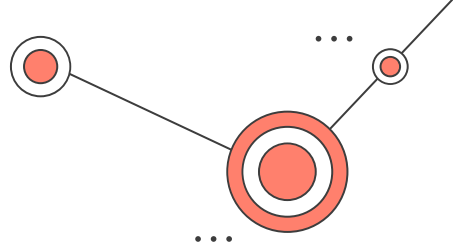
👉 What if all words could look at each other at the same time?



With Attention

- We solved the bottleneck problem...
 - **but we still process words one by one (sequentially).**
- Now imagine every word doing this with every other word...

👉 **Self-attention -> Transformer**



Transformer Architecture

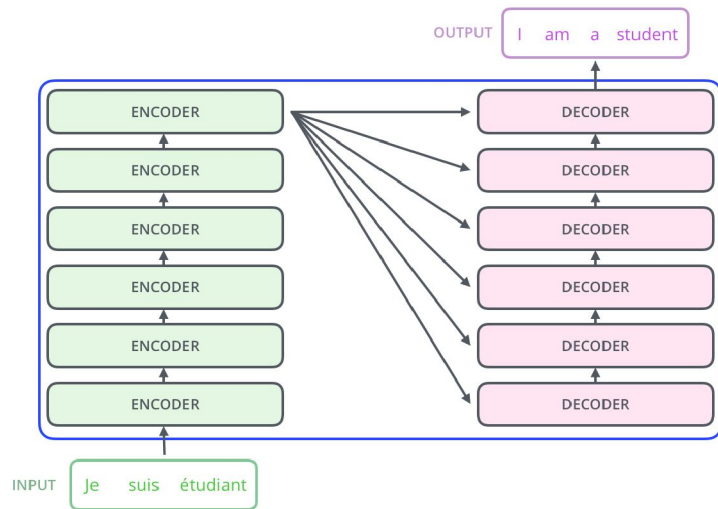


Image: <https://jalammar.github.io/illustrated-transformer/>

Vaswani et al. "Attention is all you need." NIPS. 2017.

Encoder

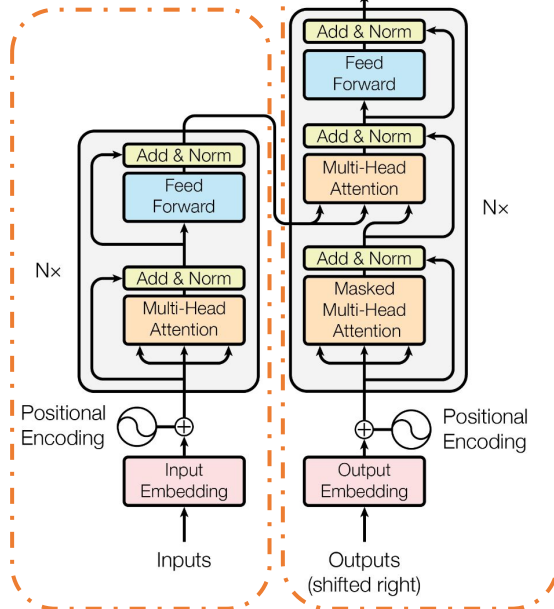


Figure 1: The Transformer - model architecture.

2017

Decoder

Transformer Architecture

Tokenization & Embedding

Embedding

Write	→	2.13	-0.42	...	-1.03
A	→	0.91	0.56	...	0.23
story	→	-1.56	1.34	...	0.14
.	→	1.42	-1.32	...	-2.41

In general embeddings send every word (token) to a long list of numbers.

Image: <https://medium.com/@amanatulla1606/transformer-architecture-explained-2c49e2257b4c>

Tokenizer: Byte-Pair Encoding (BPE)

Similar: WordPiece and SentencePiece

Summary of the tokenizers

Vaswani et al. "Attention is all you need." NIPS. 2017.

2017

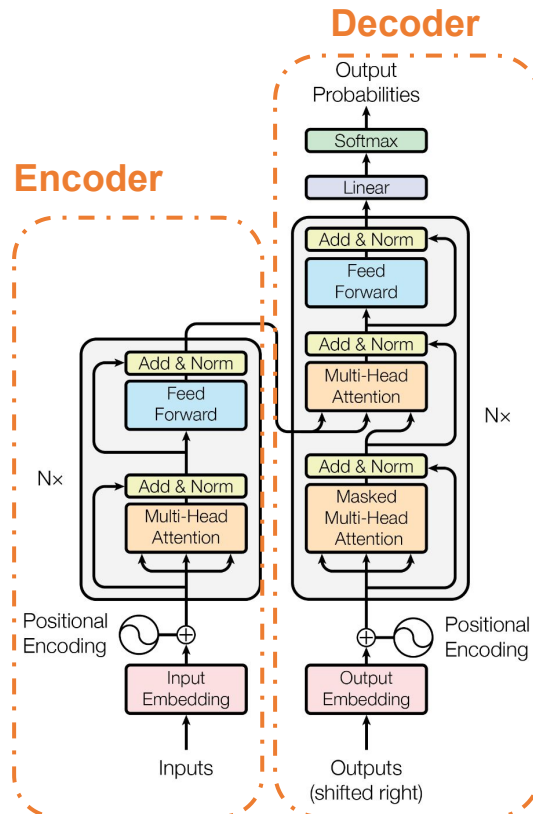


Figure 1: The Transformer - model architecture.

Transformer Architecture

Positional Encoding



Positional encoding adds a positional vector to each word, in order to keep track of the positions of the words.

Image: <https://medium.com/@amanatulla1606/transformer-architecture-explained-2c49e2257b4c>

Caveat: "I'm not sad, I'm happy" vs. "I'm not happy, I'm sad"

Vaswani et al. "Attention is all you need." NIPS. 2017.

2017

Encoder

Decoder

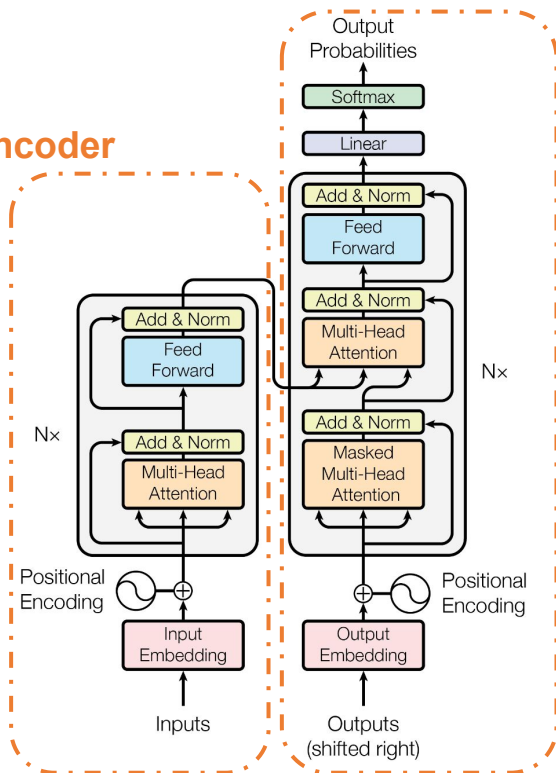


Figure 1: The Transformer - model architecture.

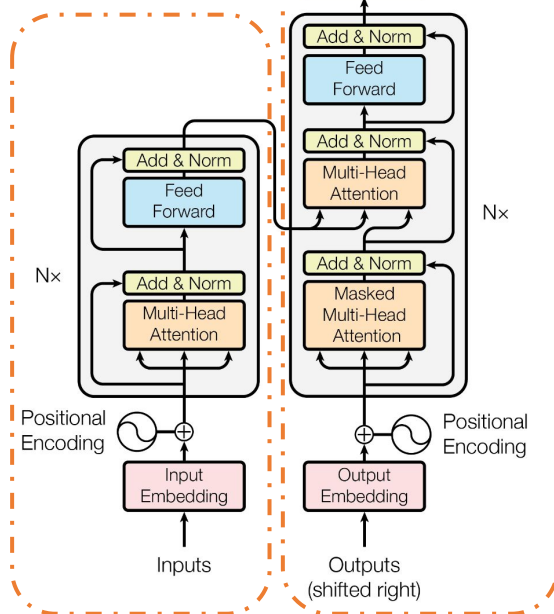
Transformer Architecture

Attention mechanisms in Transformers

- We have already seen “Encoder-Decoder” attention, Bahdanau et al. (2014)
- Bi-directional self-attention (Encoder)
 - Words in the same sentence look both at previous & future words
- Causal attention (a.k.a self-attention)
 - Each token “looks” at other tokens in the past of **the same sentence**

Vaswani et al. "Attention is all you need." NIPS. 2017.

Encoder

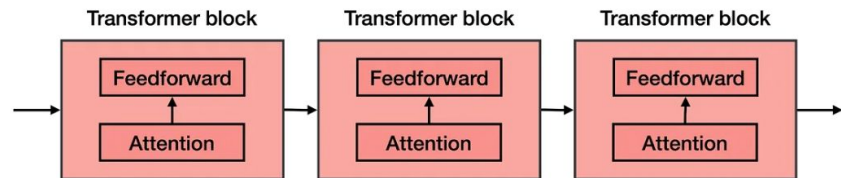


Decoder

Figure 1: The Transformer - model architecture.

Transformer Architecture

Transformer Block



The transformer is a concatenation of many transformer blocks. Each one of these is composed by an attention component followed by a feedforward component (a neural network).

Image: <https://medium.com/@amanatulla1606/transformer-architecture-explained-2c49e2257b4c>

Vaswani et al. "Attention is all you need." NIPS. 2017.

2017

Decoder

Encoder

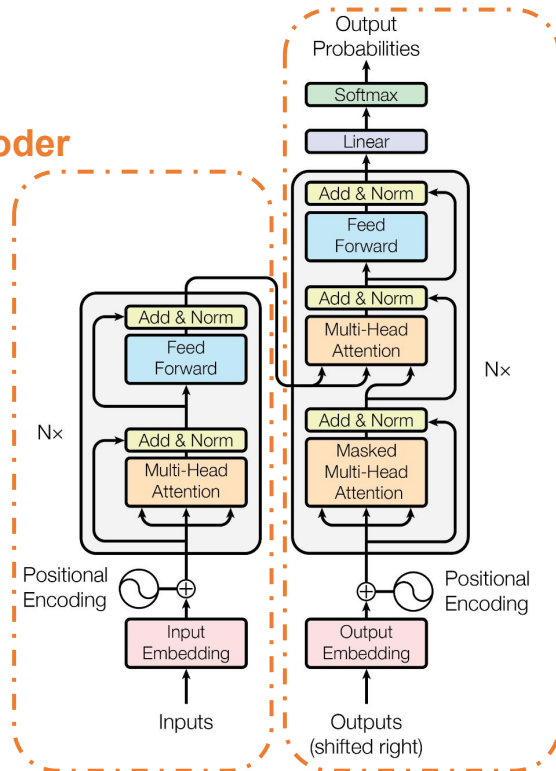


Figure 1: The Transformer - model architecture.

Transformer Architecture

Multi-Head Attention: Two heads are better than one

Split Q, K and V matrices into “heads”

$Q_1 (T \times H)$	$Q_2 (T \times H)$	$Q_{12} (T \times H)$
$\begin{bmatrix} 0.07 & 0.04 & \dots & 0.64 \\ 0.07 & 0.07 & \dots & 0.36 \\ 0.08 & 0.06 & \dots & 0.44 \\ 0.04 & 0.04 & \dots & 0.34 \end{bmatrix}$	$\begin{bmatrix} 0.19 & 0.18 & \dots & 0.39 \\ 0.20 & 0.19 & \dots & 0.28 \\ 0.20 & 0.19 & \dots & 0.31 \\ 0.18 & 0.18 & \dots & 0.27 \end{bmatrix}$	$\begin{bmatrix} 0.27 & 0.24 & \dots & 0.64 \\ 0.27 & 0.27 & \dots & 0.36 \\ 0.28 & 0.26 & \dots & 0.44 \\ 0.24 & 0.24 & \dots & 0.34 \end{bmatrix}$

12 heads of size 64 each

No. of words (T) = 4 No. of heads = 12
Embedding size (C) = 768 Head size (H) = $C/12 = 64$

Image: <https://prvnsmph.github.io/animated-transformer/>

2017

Decoder

Encoder

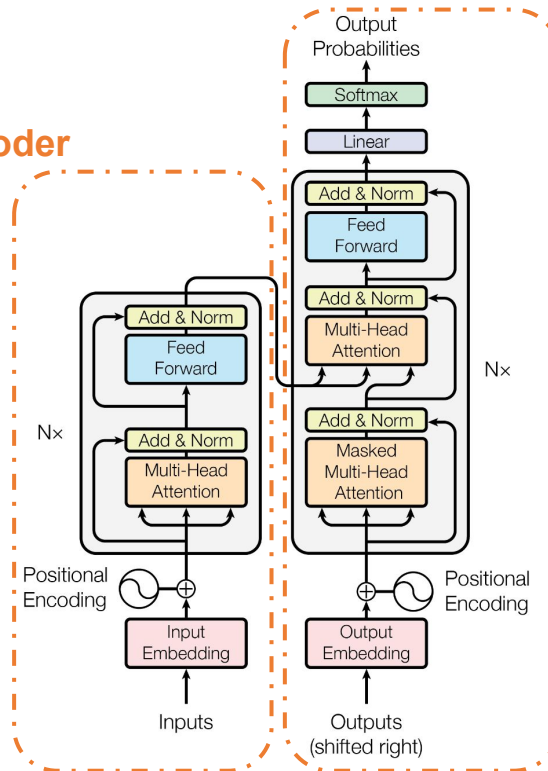


Figure 1: The Transformer - model architecture.

Transformer Architecture

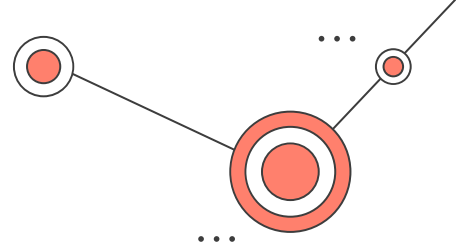
How it is different from seq-to-seq attention?

- 1) **Problem:** Word-sense disambiguation
- 2) Same word - different meanings based on context
 - a) **Sentence 1:** The **bank** of the river.
 - b) **Sentence 2:** Money in the **bank**.

Solution: Attention

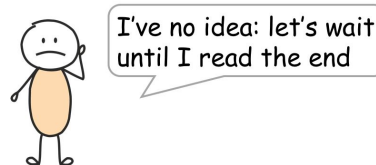
- 1) Self-Attention
- 2) Multi-Head Attention

2017



I arrived at the **bank** after crossing thestreet? ...river?

What does **bank** mean in this sentence?



RNNs

$O(N)$ steps to process a sentence with length N

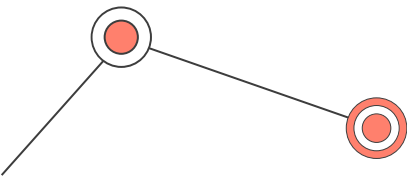


Transformer

Constant number of steps to process any sentence

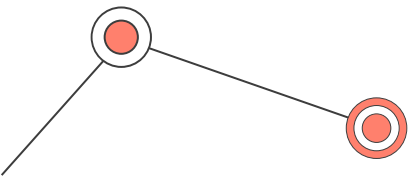
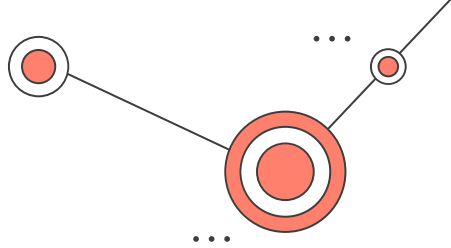
Vaswani et al. "Attention is all you need." NIPS. 2017.

Image: https://lena-voita.github.io/nlp_course/seq2seq_and_attention.html

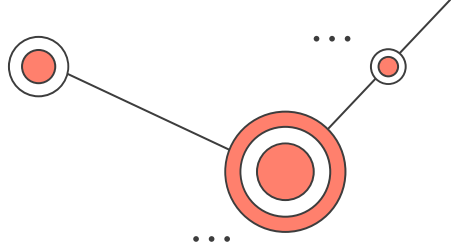


From Transformer to Modern Language Models

- Transformer = **general architecture**
- Different setups → different models:
 - **Encoder-only** → BERT → understanding
 - **Decoder-only** → GPT → generation
 - **Encoder–Decoder** → T5 → transformation

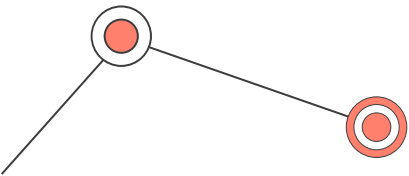


From Transformer to Modern Language Models



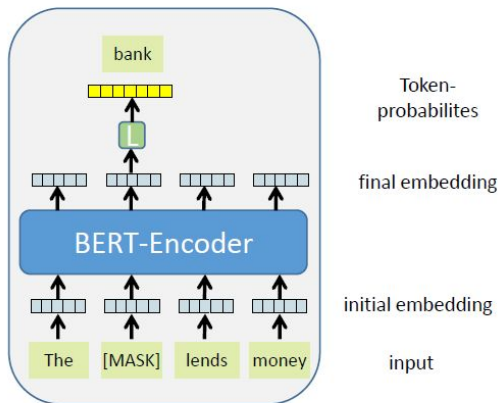
- Transformer = **general architecture**
- Different setups → different models:
 - **Encoder-only** → BERT → understanding
 - **Decoder-only** → GPT → generation
 - **Encoder–Decoder** → T5 → transformation

👉 Same building blocks, different usage



From Transformer to Modern Language Models

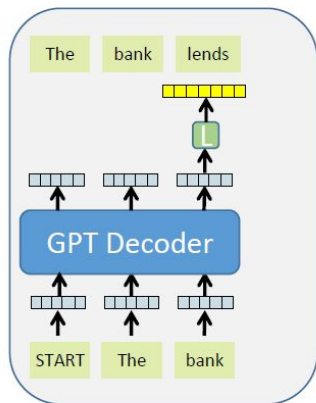
Encoder-based



BERT: Prediction of masked tokens.

- Finetuning for:
Recognition of named entities,
sentiment analysis, ...

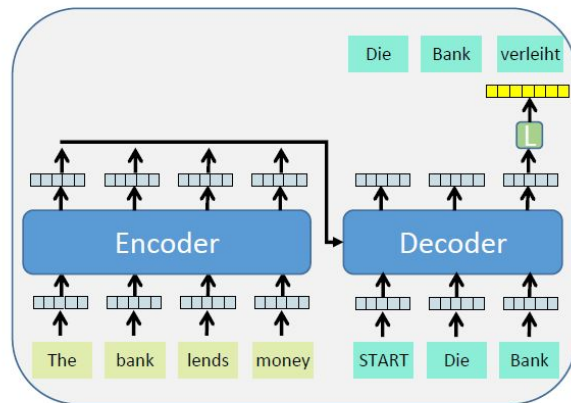
Decoder-based



Language model (GPT): Prediction of next tokens

- Self-Attention for **prior** tokens
- Adaptation via prompting

Encoder Decoder-based



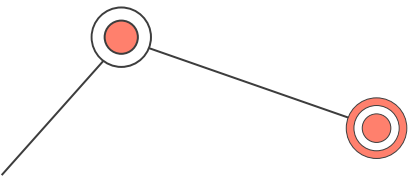
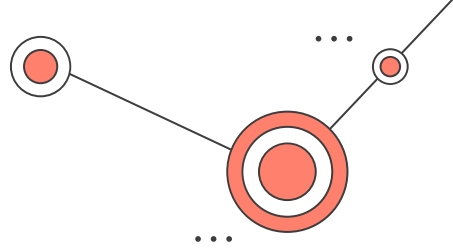
Transformer: Translation into another language or sequence type

- Cross-Attention to input embeddings
- Self-Attention for **prior** translated tokens

Paaß & Giesselbach, 2023

How Do These Models Learn?

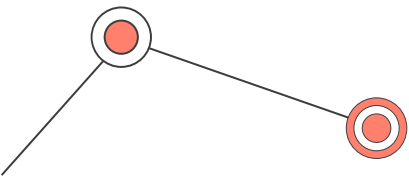
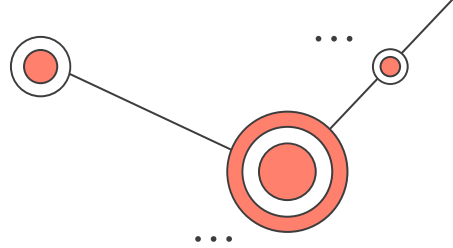
- No manual labels → learn from **raw text**
- **Self-supervised learning (learning from data itself)**
 - Early example: **Word2Vec**
 - Modern models:
 - **Next-word** prediction (GPT)
 - **Masked (missing) words** prediction (BERT)



How Do These Models Learn?

- No manual labels → learn from **raw text**
- **Self-supervised learning (learning from data itself)**
 - Early example: **Word2Vec**
 - Modern models:
 - **Next-word** prediction (GPT)
 - **Masked (missing) words** prediction (BERT)

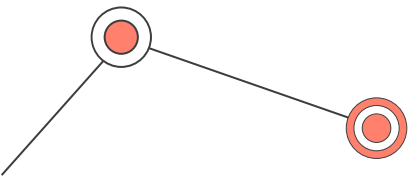
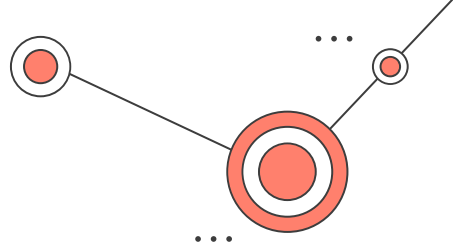
👉 Learning by predicting language patterns



Why Did These Models Become So Powerful?

- More **data**
- Bigger **models**
- More **compute**

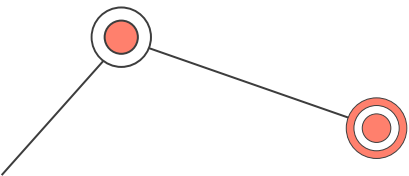
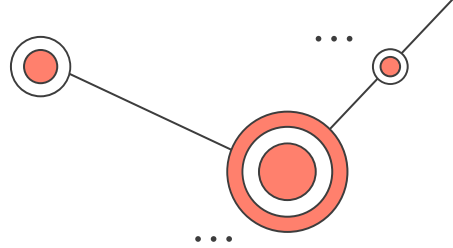
👉 Better performance with scale (unlike earlier AI approaches)

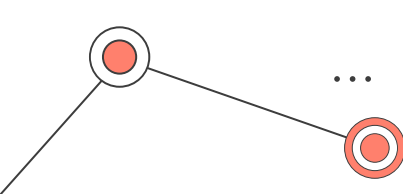


How Do We Use These Models?

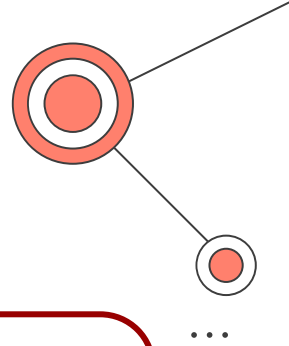
- Fine-tuning → adapt to specific tasks
- Prompting / In-context learning → no training needed

👉 One model → many tasks





The Building Blocks of Foundation Models



01

Transformer Architecture

- Better architecture
- Scalability

02

Self-Supervised Learning

- Training on massive amounts of unlabelled data
- Easily crawled from the Web

03

Transfer Learning

- Pre-training & fine-tuning (adaptation)

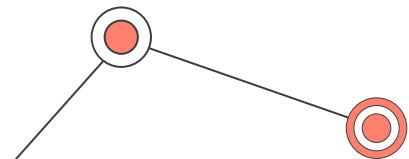
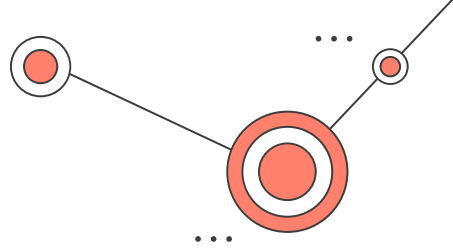
04

Computing on Large-Scale

- Large-scale (models and dataset)
- Inference with large-scale models

Self-Supervised Learning

- Training the model using large amounts of data itself, i.e. without human annotations
- GPT training objective: Predicting the next word
- Typically used for **pre-training**
- Important examples in NLP (before LLMs)
 - **Word2Vec**: CBOW or skip-gram Word2Vec
 - **ELMo**: Bi-directional language modelling ELMO
- **LLM examples**: BERT, GPT, T5 etc.



Self-Supervised Learning

Pre-Training Objectives

- LM (Language Model), e.g. ELMo, GPT - predicting the next word
- MLM (Masked Language Modelling), e.g. BERT - predicting the randomly masked (hidden) words

LM language model

$$L_{LM} = - \sum_{t=1}^T \log p(x_t | \mathbf{x}_{<t})$$

e.g. ELMo, GPT

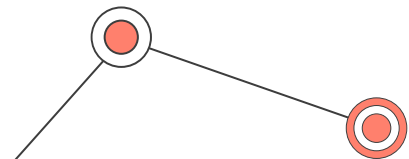
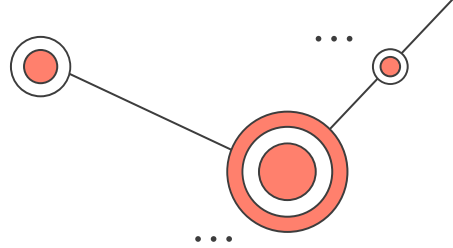
MLM masked language model

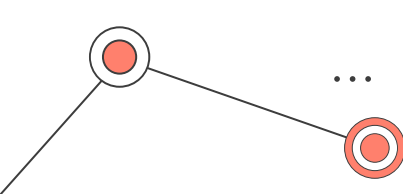
$$L_{MLM} = - \sum_{t \in m(\mathbf{x})} \log p(x_t | \tilde{\mathbf{x}})$$

$m(\mathbf{x})$ contains the indices of masked tokens in $\mathbf{x}$. In $\tilde{\mathbf{x}}$ the masked tokens are replaced by *MASK*, e.g. BERT,

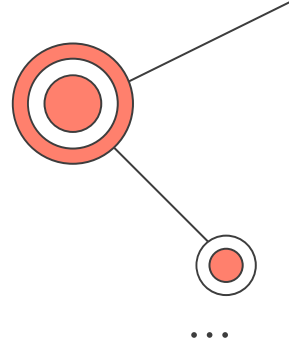
Improving Language Understanding by Generative Pre-Training

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding





The Building Blocks of Foundation Models



01

Transformer Architecture

- Better architecture
- Scalability

02

Self-Supervised Learning

- Training on massive amounts of unlabelled data
- Easily crawled from the Web

03

Transfer Learning

- Pre-training & fine-tuning (adaptation)

04

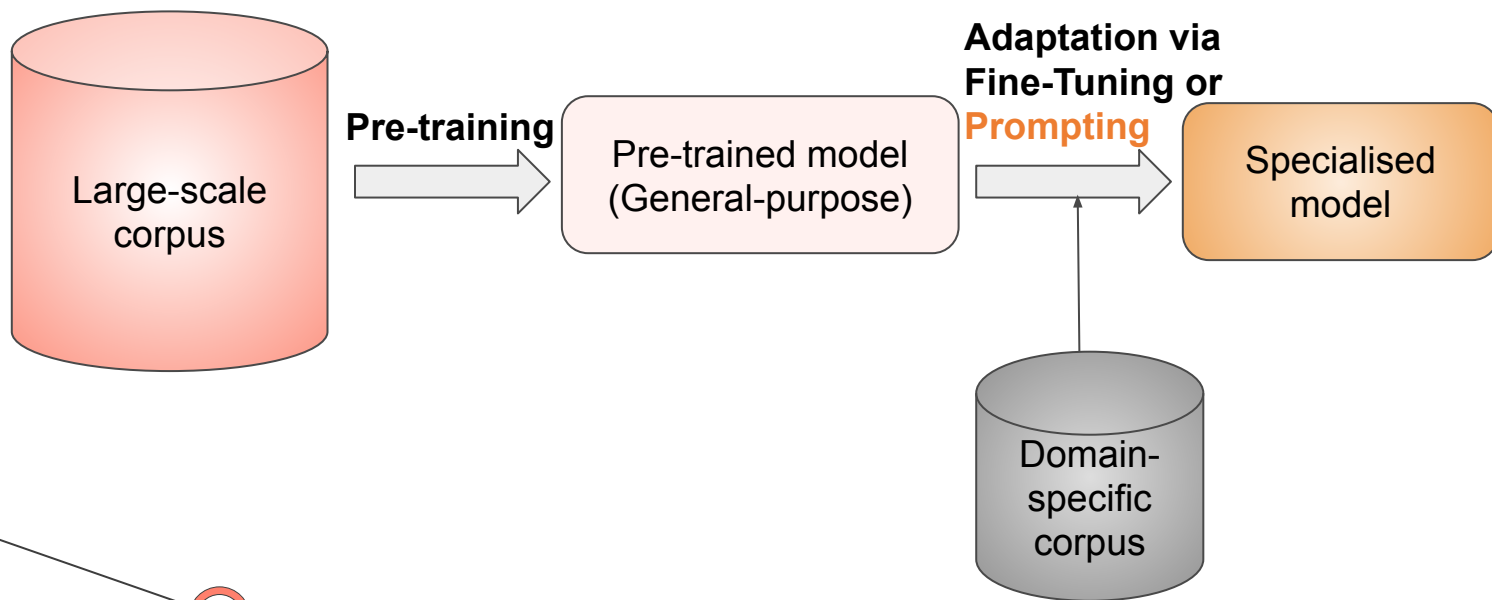
Computing on Large-Scale

- Large-scale (models and dataset)
- Inference with large-scale models

Transfer Learning

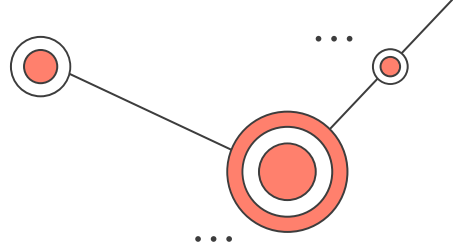
New Paradigm : Pre-train and fine-tuning or prompting

- Train a model on large amounts of data (pre-training)
- Fine-tune it on a smaller domain-specific data



Prompting LLMs

Concrete Examples



Zero-shot:

Task description:
Convert English to French

Prompt:
cheese=>

One-shot:

Task description:
Convert English to French

Example:
Sea otter=> loutre de maar

Prompt:
cheese=>

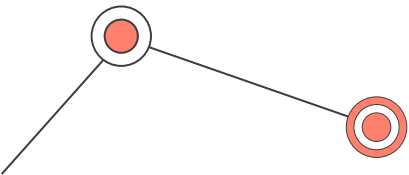
Few-shot:

Task description:
Convert English to French

Examples:
Sea otter=> loutre de maar
Beach => la plage

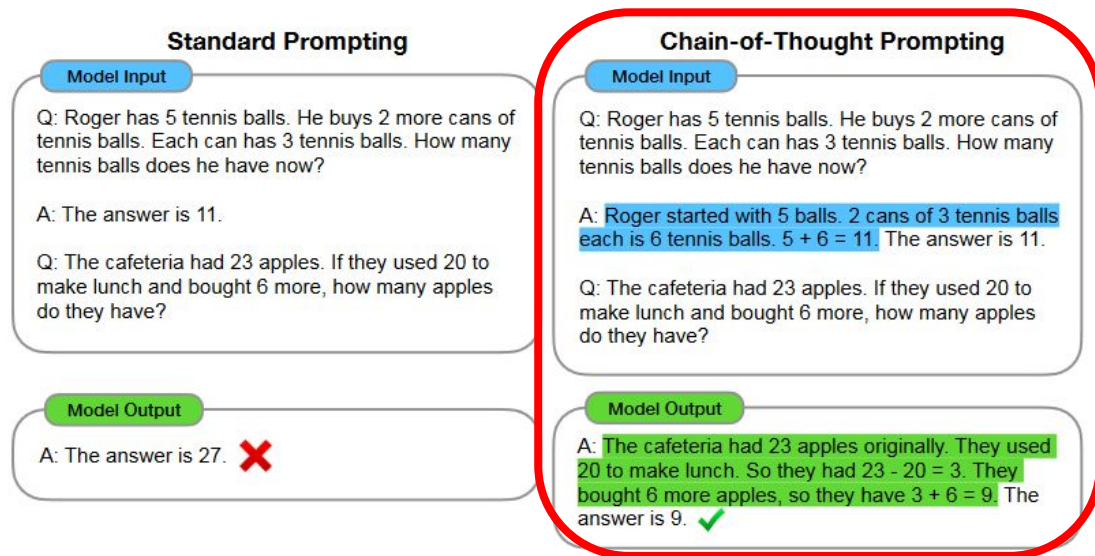
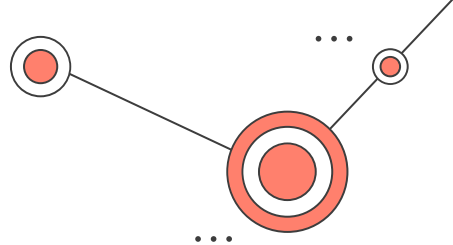
Prompt:
cheese=>

[Dong et al. A Survey on In-context Learning, 2024](#)



Instruction Formatting

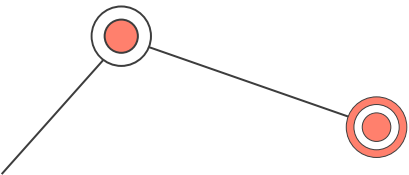
Chain-of-Thought Prompting (CoT)

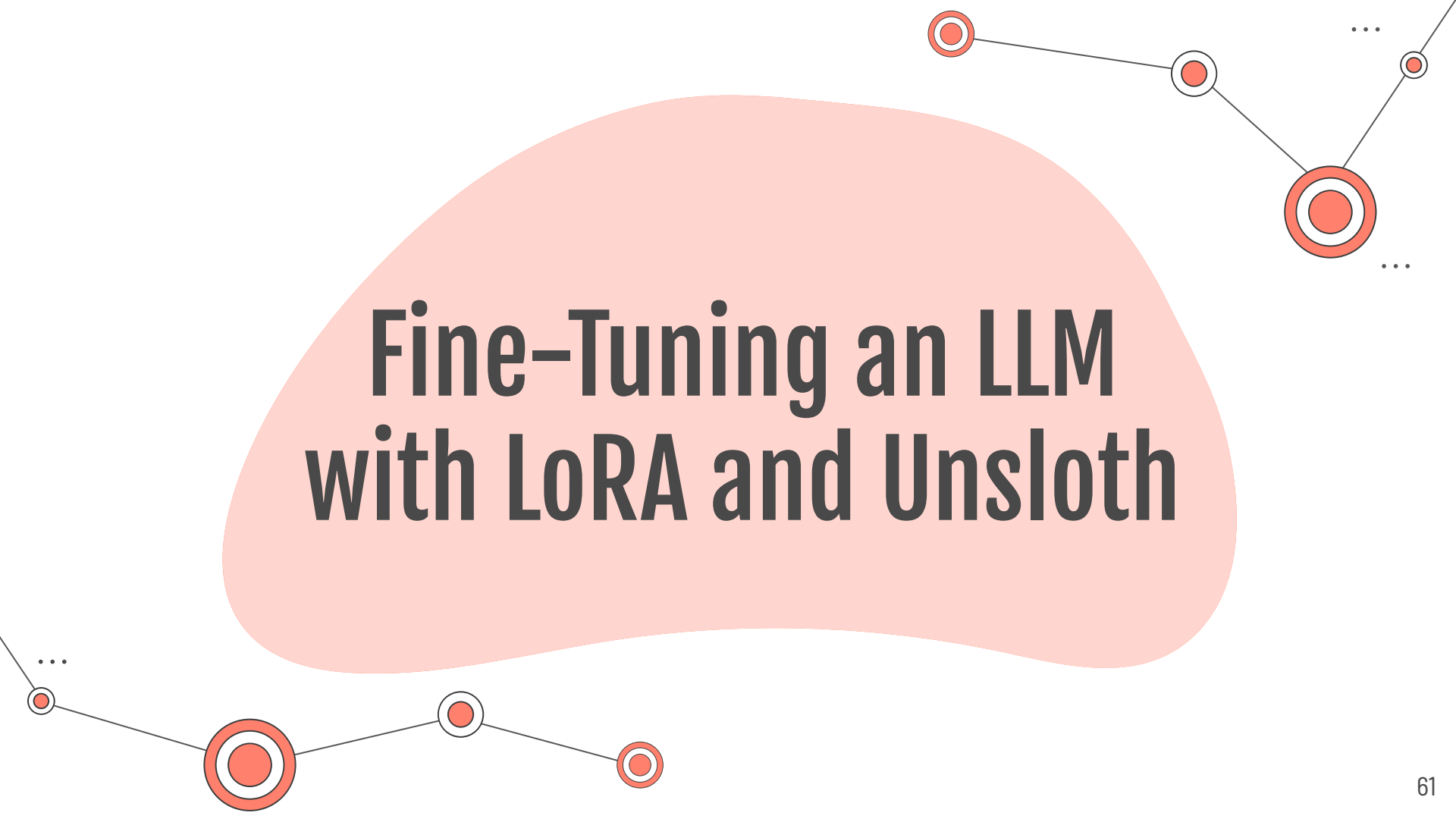


- **Zero-shot CoT:** Think step by step.
- **Few-shot CoT:** Show examples with reasoning, then ask a new question

Figure 1: Chain-of-thought prompting enables large language models to tackle complex arithmetic, commonsense, and symbolic reasoning tasks. Chain-of-thought reasoning processes are highlighted.

[Wei et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. 2022.](#)





Fine-Tuning an LLM with LoRA and Unsloth

Load the Base Model

```
from unsloth import FastLanguageModel
import torch

max_seq_length = 15000
dtype = None
load_in_4bit = True

model, tokenizer = FastLanguageModel.from_pretrained(
    model_name="unsloth/tinyllama-bnb-4bit",
    max_seq_length=max_seq_length,
    dtype=dtype,
    load_in_4bit=load_in_4bit,
)
```

- Load a pretrained LLM in 4-bit quantized format
- Less GPU memory usage

Add LoRA Adapters

```
model = FastLanguageModel.get_peft_model(  
    model,  
    r=16,  
    target_modules=[  
        "q_proj", "k_proj", "v_proj", "o_proj",  
        "gate_proj", "up_proj", "down_proj",  
    ],  
    lora_alpha=16,  
    lora_dropout=0,  
    bias="none",  
    use_gradient_checkpointing="unsloth",  
    random_state=3407,  
)
```

- Instead of updating the full model, we use LoRA
- Train only LoRA adapter weights

Load CSV Data

```
from datasets import Dataset
import pandas as pd

df =
pd.read_csv("/content/drive/MyDrive/Projects/feminici
des/main_dataset_for_ft.csv")

dataset = Dataset.from_pandas(df)
dataset = dataset.map(formatting_prompts_func,
batched=True)
```

- Load the labeled dataset
- Convert it into huggingface Dataset format

Training

```
from trl import SFTTrainer
from transformers import TrainingArguments
from unsloth import is_bfloat16_supported

trainer = SFTTrainer(
    model=model, tokenizer=tokenizer, train_dataset=dataset,
    dataset_text_field="text", max_seq_length=max_seq_length, packing=False,
    args=TrainingArguments(per_device_train_batch_size=2,
        gradient_accumulation_steps=4, warmup_steps=5, max_steps=60,
        learning_rate=2e-4, fp16=not is_bfloat16_supported(),
        bf16=is_bfloat16_supported(), logging_steps=1, optim="adamw_8bit",
        weight_decay=0.01, lr_scheduler_type="linear", seed=3407,
        output_dir="/content/drive/MyDrive/Projects/feminicides/output", ))
trainer_stats = trainer.train()
```

- Set the training parameters
- Start supervised finetuning

Inference

- We give the model new text
- Ask it to predict the label

```
FastLanguageModel.for_inference(model)

inputs = tokenizer(
    [alpaca_prompt.format("Classifica il seguente testo come 'focus_killer' o
    'focus_victim' o 'no_focus'.", #instruction

"Era il 2002, e scrivevo, # input
"", # output - leave this blank for generation!
)
],return_tensors="pt",).to("cuda")

outputs = model.generate(**inputs,max_new_tokens=64,use_cache=True,)

prediction = tokenizer.batch_decode(outputs)[0]
print(prediction)
```

References

1. [Vaswani, Ashish, et al. "Attention is all you need." Advances in neural information processing systems 30 \(2017\).](#)
2. [Cho, Kyunghyun, et al. "Learning phrase representations using RNN encoder-decoder for statistical machine translation." arXiv preprint arXiv:1406.1078 \(2014\).](#)
3. [Sutskever, Ilya, Oriol Vinyals, and Quoc V. Le. "Sequence to Sequence Learning with Neural Networks."](#)
4. [Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio. "Neural machine translation by jointly learning to align and translate." arXiv preprint arXiv:1409.0473 \(2014\).](#)
5. [Luong, Minh-Thang, Hieu Pham, and Christopher D. Manning. "Effective approaches to attention-based neural machine translation." arXiv preprint arXiv:1508.04025 \(2015\).](#)
6. [Summary of the tokenizers](#)
7. [Peters, Matthew E., et al. "Dissecting contextual word embeddings: Architecture and representation." arXiv preprint arXiv:1808.08949 \(2018\).](#)
8. [Mikolov, Tomas, et al. "Efficient estimation of word representations in vector space." arXiv preprint arXiv:1301.3781 \(2013\).](#)
9. [Devlin, Jacob, et al. "Bert: Pre-training of deep bidirectional transformers for language understanding." Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 \(long and short papers\). 2019.](#)
10. [Radford, Alec, et al. "Improving language understanding by generative pre-training." \(2018\).](#)